Socket Programming Project

LAN Chat & File Sharing (Command-Line)

Assignment Brief • Using TCP and UDP Sockets

Item	Details
Team size	2-3 students
Language	Python, Java, C/C++, … (**Use the programming language's native Socket API for communication**)
Interface	Command-line (console) or GUI
Environment	2+ computers on the same LAN/Wi-Fi, or multiple instances on one machine

1. Overview

You will build a peer-to-peer chat application that runs in the console. Every user runs the same program. The program automatically discovers other users on the local network, shows who is online, lets the user pick someone to chat with, and supports sending files.

The project deliberately uses **both** transport protocols, each for the job it is best at:

- **UDP** – for discovery and status (fast, connectionless, losing a packet is acceptable).
- **TCP** – for chat messages and file transfer (reliable, ordered, nothing may be lost).

2. Learning Objectives

After completing this project, students should be able to:

1. Explain the differences between TCP (connection-oriented, reliable byte stream) and UDP (connectionless, unreliable datagrams).
2. Create, bind, and use TCP and UDP sockets, including UDP broadcast.
3. Design a simple application-layer protocol with message framing over a TCP stream.
4. Handle multiple concurrent connections using threads (or select/poll).
5. Build a responsive console interface where messages can arrive while the user is typing.
6. Handle errors: disconnections, timeouts, and partially received data.

3. Functional Requirements

3.1 Peer Discovery (UDP)

- Each peer broadcasts a presence message every 3 seconds on a fixed UDP port (e.g. 9999).
- Each peer listens on that port and maintains a list of online users (name, IP, TCP port, status).
- A user not heard from for 10 seconds is removed from the list (timeout).
- The console shows a notification when a user comes online or goes offline.

3.2 Online Status (UDP)

- Users can set their status: online or away.

- Status is included in the presence broadcast and displayed in the online list.

3.3 Chat (TCP)

- A user selects an online peer; the program opens a TCP connection to that peer.
- Users can send messages at any time, one message per line, as many lines as they want.
- Incoming messages are displayed immediately, even while the user is typing.
- Messages from a peer who is not the current chat partner are stored and shown as a notification with an unread count.

3.4 File Transfer (TCP)

- A user can send a file to the current chat partner.
- The transfer shows a progress indicator (percentage).
- Received files are saved in a downloads/ folder.
- Chatting must keep working while a file is being transferred (run the transfer in its own thread).

4. Command-Line Interface

4.1 Commands

Normal text is sent as a chat message. Text starting with / is a command.

Command	Description
`/list`	Show online users with number, name, IP and status
`/chat <name|number>`	Open a chat with the selected user (TCP connection)
<any text>	Send a message to the current chat partner
`/send <file>`	Send a file to the current chat partner
`/leave`	Close the current chat
`/status <online|away>`	Change your status (sent via UDP)
`/help`	Show all commands
`/quit`	Exit the program

4.2 Example Session

```
$ python lanchat.py --name Quan
[*] Listening on TCP port 5000
[*] Broadcasting presence on UDP port 9999
> /list
  1. Minh   (192.168.1.12)  online
  2. Lan    (192.168.1.15)  away
  3. Huy    (192.168.1.20)  online
> /chat Minh
[*] Connected to Minh (192.168.1.12:6000)
[Quan] Hi Minh, are you ready for the demo?
[Minh] Yes! Send me the slides
[Quan] I check that
[Quan] it's not enough content
[Quan] you can check it again
[*] New message from Huy (1 unread)
> /send slides.pdf
[*] Sending slides.pdf (2.4 MB) ... 100%  Done.
> /quit
```

```
Quan's console

> /chat Minh
[*] Connected to Minh
(192.168.1.12:6000)
[Quan] Minh, did you get the slides?
[Quan] need to check before 3pm
>
```

```
Minh's console (current partner = Huy)

[Huy] the router config is done
[Minh] ok send me the screenshot
[*] New message from Quan (1 unread)
[*] New message from Quan (2 unread)          <- or update the same line
[Huy] sending now
> /chat Quan                                   <- Minh switches
[*] Switched to Quan – 2 unread:
[Quan] Minh, did you get the slides?
[Quan] need to check before 3pm
[Minh] yes, checking now
>
```

4.3 Typing While Messages Arrive

The console has no separate boxes, so the program must simulate them: the last line of the screen is always the input line, and everything above it is the chat history.

When a message arrives while the user is typing, the program must:

1. Erase the current input line (\r followed by the ANSI code \033[K).
2. Print the incoming message on that line, followed by a newline.
3. Redraw the prompt "> " together with the text the user had already typed.

Before and after a message from Minh arrives:

```
[Quan] it's not enough content
> you can che_                  <- user is typing

[Quan] it's not enough content
[Minh] ok, which part?          <- inserted above
> you can che_                  <- typed text is preserved
```

Implementation options: (a - ***option_a_prompt_toolkit.py***) the prompt_toolkit library with patch_stdout(), which handles redrawing automatically; or (b - ***option_b_manual_redraw.py, chat_redraw.cpp***) reading keys one at a time (msvcrt.getwch() on Windows), keeping the typed text in a buffer, and redrawing manually. A lock must protect screen output so two threads never print at the same time.

5. Technical Requirements

5.1 Thread Model

Thread	Waits for	Responsibility
Main	Keyboard	Read input, run commands, send messages
UDP broadcaster	Timer (3 s)	Send presence message
UDP listener	UDP packets	Update the online-users list
TCP acceptor	New connections	Start a receiver thread for each peer
TCP receiver (one per peer)	That peer's socket	Display messages, save files
File sender (per transfer)	—	Send a file without blocking the chat

Shared data (online-users list, chat histories, unread counters) must be protected with a lock.

5.2 Protocol example (create your protocol by yourself)

UDP presence message (text, one datagram):

```
HELLO|<name>|<tcp_port>|<status>
e.g.  HELLO|Quan|5000|online
```

TCP framing – TCP is a byte stream with no message boundaries, so every message must start with a header that tells the receiver its type and length:

```
MSG|<length>|<utf-8 text>
FILE|<filename>|<size_in_bytes>|<raw file bytes>
BYE|
```

The receiver must read exactly <length> or <size> bytes (loop on recv() until all bytes arrive).

5.3 Error Handling

- Peer closes the connection or crashes: show a notice and return to the main prompt.
- Connecting to a user who has gone offline: show an error, do not crash.
- File not found, or transfer interrupted: show an error and delete the partial file.
- Invalid commands: show a short help message.

6. Milestones

No.	Task	Checkpoint
1	TCP echo client/server; UDP send/receive test	Demo both programs
2	UDP discovery, online list, /list, /status	Peers appear and disappear on 2+ machines
3	TCP chat with /chat, /leave, framing, threads (select/poll)	Two users chat in both directions
4	Typing-while-receiving UI; file transfer with progress	Send a file while chatting
5	Error handling, testing, report, demo	Final submission

7. Deliverables

1. Source code with a README (how to install, run, and test).
2. A report (5–10 pages): architecture, thread model, protocol format, problems faced and solutions.
3. Wireshark screenshots showing the UDP broadcast packets and the TCP three-way handshake, with a short explanation.
4. A live demo (10 minutes) on at least two computers.

8. Grading Rubric

Criteria	Points	Weight
UDP discovery, online list, status	15	15%
TCP chat (send/receive at any time, multiple peers)	20	20%
File transfer with progress, saved correctly	15	15%

Criteria	Points	Weight
Console UI: typing is not broken by incoming messages Or your GUI	15	15%
Protocol design and message framing	10	10%
Error handling and stability	10	10%
Report	10	10%
Demo and code quality	5	5%
Total	100	100%

9. Hints and Resources

- Testing on one machine: run several instances with different TCP ports (--port 5001, --port 5002).
- Use netcat (nc) as a quick TCP/UDP test client.
- **Testing several instances on one PC (udp_broadcast_demo.py)**
    a. set SO_REUSEADDR/SO_REUSEPORT before binding, and bind to "";
    b. offline, broadcast to 127.255.255.255.
- Allow Python through the Windows Firewall, or broadcasts will not be received.
- Reading: Beej's Guide to Network Programming; the Python socket and threading documentation.